School Face Attendance System

API Reference for Web Integration Team

Version 3.0 | May 2026

This document describes the REST and WebSocket APIs exposed by the School Face Attendance backend. It is intended for the website development team who will build the UI that consumes these endpoints. The backend runs on Python / FastAPI and stores data in PostgreSQL.

Method Legend

Method	Colour
GET	Green - read data
POST	Amber - send data
DELETE	Red - remove data
WS	Purple - WebSocket

1. System Overview

How it works end-to-end

The School Face Attendance System is a Python / FastAPI backend that uses computer-vision AI to identify students from a live camera and automatically record their attendance in a PostgreSQL database. Your website integrates with it through the REST and WebSocket APIs documented in this file.

Step-by-step flow:

1. Camera sends a live video frame to the server (via WebSocket or HTTP).
2. SCRFD 500M face detector locates every face in the frame.
3. MiniFASNet V2 liveness check rejects photos / screen replays.
4. ArcFace R50 converts the aligned face crop into a 512-number vector (embedding).
5. The embedding is compared against all stored student embeddings using cosine similarity. The closest match above a confidence threshold is identified.
6. Attendance is written to the database (one record per student per day).
7. The website reads attendance and statistics through the Dashboard APIs.

Architecture

Camera / Browser	`--> WebSocket /ws/camera OR GET /api/camera/stream`
Edge device (IP cam)	`--> POST /api/camera/process-frame`
Registration form	`--> POST /api/register`
Attendance view	`--> GET /api/attendance`
Dashboard / Stats	`--> GET /api/stats`

Base URL

All HTTP endpoints are relative to:

```
http://<server-host>:8000
```

Authentication

No authentication is required for this version. CORS is open (allow-all). If the system is exposed to the internet, place it behind a reverse proxy (nginx / Caddy) and add your own auth layer.

2. Face Embedding Technology

What is a face embedding?

A face embedding is a compact numerical representation of a person's face. The AI model converts a face image into a list of 512 numbers (a vector). People who look similar produce vectors that are close together; different people produce vectors that are far apart. This is how the system tells students apart without storing any raw photos in the database.

Model used

Property	Value
Model name	ArcFace R50 (InsightFace buffalo_l)
Input	112 x 112 pixel aligned face crop (BGR)
Output	512-dimensional float32 vector, L2-normalised
Similarity metric	Cosine similarity (range -1.0 to 1.0; higher = more similar)
Threshold	0.55 (configurable in .env via SIMILARITY_THRESHOLD)
Memory per student	512 x 4 bytes = 2 048 bytes (~2 KB)
Liveness check	MiniFASNet V2 ONNX (rejects printed photos / screen replays)

How embeddings are stored

After registration the 512 float32 values are serialised to raw bytes (numpy tobytes()) and saved in the PostgreSQL embeddings table as a BYTEA column. Each student has exactly one row in that table containing their averaged embedding across all registration photos. No face images are stored in the database.

```
-- PostgreSQL embeddings table
CREATE TABLE embeddings (
id SERIAL PRIMARY KEY,
student_id INT NOT NULL REFERENCES students(id) UNIQUE,
created_at TIMESTAMP DEFAULT NOW(),
sample_count INT DEFAULT 0, -- number of photos used
vector BYTEA NOT NULL -- 512 x float32 = 2048 bytes
);
```

How embeddings are retrieved for attendance verification

When a face is detected during live recognition the following steps happen entirely in the backend -- the web team does not need to handle any of this:

1. All student embeddings are loaded from PostgreSQL into memory as numpy arrays.

2. The live face embedding is compared to every stored embedding using cosine similarity: score = stored_vector dot live_vector

3. The student with the highest score is selected. If that score is below the threshold (0.55) the face is reported as Unknown.

4. If a match is found, one attendance record is written for today (duplicates are silently ignored -- each student is marked once per day).

> *i Re-registration: submitting new photos for an existing student replaces their stored embedding (ON CONFLICT DO UPDATE). The old embedding is overwritten automatically.*

3. Live Camera API

Two ways to receive the live camera feed. Use whichever fits your frontend.

GET `/api/camera/stream`

MJPEG live feed with face-detection bounding boxes and name labels drawn on each frame. The simplest integration: drop an <img> tag pointing at this URL and the browser streams the video automatically. This endpoint is view-only -- it does NOT mark attendance.

i Content-Type: multipart/x-mixed-replace; boundary=frame

i Usage in HTML: <img src="http://server:8000/api/camera/stream">

i The stream runs until the client disconnects. Each JPEG frame has face boxes and confidence labels overlaid by the server.

WS `/ws/camera`

WebSocket live feed. The server pushes two types of messages: Binary frames -- raw JPEG bytes to display as video. Text frames -- JSON attendance events when a known student is recognised. Attendance IS marked automatically through this endpoint (once per student per 30 seconds).

Response

```
// Binary message: render as video frame
blob -> ImageBitmap or <canvas> draw
// Text message: attendance event (JSON)
{
"type": "attendance",
"name": "Ali Abbas",
"status": "marked", // or "already_marked"
"confidence": 0.91
}
```

i Connect with: new WebSocket("ws://server:8000/ws/camera")

i The server opens the webcam on its own machine. Suitable when the camera is physically connected to the server PC.

i Cooldown: the same student triggers at most one attendance event every 30 seconds.

POST `/api/camera/process-frame`

Submit a single JPEG frame captured by an external camera (IP camera, Raspberry Pi, etc.). The server detects faces, identifies students, marks attendance, and returns the results. Intended for edge devices that poll on their own schedule.

Parameters

Parameter	Type	Required	Description
`file`	*JPEG file (multipart)*	**Yes**	The captured frame image.
`camera_id`	*string (form field)*	No	Identifier for this camera, stored in the attendance record. Default: cam_01.

Request Example

```
curl -X POST http://server:8000/api/camera/process-frame \
-F "file=@frame.jpg" \
-F "camera_id=entrance_cam"
```

Response

```
{
"faces_detected": 2,
"results": [
{ "name": "Ali Abbas", "status": "marked", "confidence": 0.91 },
{ "name": "Unknown", "status": "unknown", "confidence": 0.21 }
]
}
```

i status values: "marked" (new record), "already_marked" (duplicate today), "unknown" (face not recognised), "db_missing" (FAISS match but no DB row).

4. Student Registration API

Endpoints for registering students, listing them, and deactivating them. Registration collects face photos, computes the ArcFace embedding, and stores it in PostgreSQL.

POST /api/register

Register a new student. Upload at least 5 clear face photos. The server detects the face in each photo, computes a 512-dim embedding, averages all valid embeddings into one representative vector, and stores it. Duplicate roll numbers are rejected.

Parameters

Parameter	Type	Required	Description
`name`	*string (form field)*	**Yes**	Full name of the student.
`roll_number`	*string (form field)*	**Yes**	Unique roll / student ID. Duplicate is rejected.
`class_name`	*string (form field)*	**Yes**	Class or grade (e.g. "10A").
`section`	*string (form field)*	No	Section label (e.g. "B"). Default: empty.
`photos`	*JPEG files (multipart)*	**Yes**	One or more face photos. Minimum 5 valid face detections required across all photos.

Request Example

```
// HTML form example
<form enctype="multipart/form-data"
action="http://server:8000/api/register" method="POST">
<input name="name" type="text">
<input name="roll_number" type="text">
<input name="class_name" type="text">
<input name="section" type="text">
<input name="photos" type="file" multiple accept="image/*">
<button type="submit">Register</button>
</form>
```

Response

```
// Success
{ "success": true, "student_id": 7, "samples_used": 8 }
// Failure -- duplicate roll number
{ "success": false, "error": "Roll number already registered" }
// Failure -- too few valid face photos
{ "success": false, "error": "Only 3 valid face samples found. Need at least 5." }
```

i Upload at least 5-10 photos per student for best recognition accuracy.

i Photos should be well-lit, front-facing, with the face clearly visible.

i Re-submitting the same roll_number will be rejected. Deactivate the student first (DELETE endpoint below) then re-register to update their embedding.

GET `/api/students`

Returns the list of all active (non-deactivated) students.

Parameters

Parameter	Type	Required	Description
`class_name`	*string (query)*	No	Filter by class name. Omit to return all classes.

Response

```
[
{
"id": 3,
"name": "Ali Abbas",
"roll_number": "2024-CS-031",
"class_name": "10A",
"section": "B",
"registered_at": "2026-05-13 09:15:22"
},
...
]
```

i Deactivated students (soft-deleted) are excluded from this list.

DELETE `/api/students/{id}`

Soft-deletes a student by setting is_active = FALSE. The student's data and embedding remain in the database but they are excluded from recognition and all list endpoints. Returns 404 if the ID does not exist.

Parameters

Parameter	Type	Required	Description
`id`	*integer (path)*	**Yes**	The numeric student ID from GET /api/students.

Request Example

```
DELETE http://server:8000/api/students/3
```

Response

```
{ "success": true }
// If not found:
{ "detail": "Student not found" } // HTTP 404
```

i This is a soft delete. Data is preserved for historical attendance records.

5. Attendance / Validation API

Endpoints for reading attendance records and exporting them. Attendance is written automatically by the Camera APIs -- these endpoints are for reading and reporting only.

GET `/api/attendance`

Returns attendance records. Filter by date and/or class. Results are ordered by most recent first.

Parameters

Parameter	Type	Required	Description
`date_str`	*string YYYY-MM-DD (query)*	No	Filter to a specific date. Omit for all dates.
`class_name`	*string (query)*	No	Filter to a specific class. Omit for all classes.

Request Example

```
GET /api/attendance?date_str=2026-05-13&class_name=10A
```

Response

```
[
{
"name": "Ali Abbas",
"roll_number": "2024-CS-031",
"class_name": "10A",
"section": "B",
"date": "2026-05-13",
"marked_at": "2026-05-13 08:03:44",
"confidence": 0.91,
"camera_id": "entrance_cam"
},
...
]
```

i Each student appears at most once per date (enforced at DB level by UNIQUE(student_id, date)).

i confidence is the cosine similarity score (0.0 - 1.0) at the moment of identification.

GET /api/attendance/export

Downloads attendance for a specific date as a CSV file. Useful for generating Excel reports or printing registers.

Parameters

Parameter	Type	Required	Description
`date_str`	*string YYYY-MM-DD (query)*	**Yes**	The date to export.

Request Example

```
GET /api/attendance/export?date_str=2026-05-13
```

Response

```
// Response: Content-Type: text/csv
// Content-Disposition: attachment; filename=attendance_2026-05-13.csv
name,roll_number,class_name,section,date,marked_at,confidence,camera_id
Ali Abbas,2024-CS-031,10A,B,2026-05-13,2026-05-13 08:03:44,0.91,entrance_cam
...
```

i Columns match exactly those returned by GET /api/attendance.

6. Dashboard / Statistics API

High-level summary endpoints for building dashboards, charts, and reports.

GET `/api/stats`

Returns a summary of attendance for a given date (defaults to today). Includes total students, present, absent, attendance rate, and a per-class breakdown. Use this endpoint to power the main dashboard widgets.

Parameters

Parameter	Type	Required	Description
`date_str`	*string YYYY-MM-DD (query)*	No	Target date. Defaults to today.
`class_name`	*string (query)*	No	Scope stats to one class. Omit for school-wide totals.

Request Example

```
GET /api/stats
GET /api/stats?date_str=2026-05-13
GET /api/stats?class_name=10A
```

Response

```
{
"date": "2026-05-13",
"total_students": 120,
"present": 98,
"absent": 22,
"attendance_rate": 81.7, // percentage
"by_class": [
{ "class_name": "10A", "total": 35, "present": 30 },
{ "class_name": "10B", "total": 32, "present": 28 },
{ "class_name": "11A", "total": 53, "present": 40 }
]
}
```

i attendance_rate = (present / total_students) x 100, rounded to 1 decimal place.

i by_class lists every class that has at least one active student.

i absent = total_students - present (students with no attendance record for the date).

GET `/api/status`

Health-check endpoint. Returns whether the pipeline is loaded and how many students have registered embeddings in the database. Call this on page load to confirm the backend is reachable.

Response

```
{
"status": "running",
"mode": "heavy", // ArcFace R50
"liveness_enabled": true,
"students_in_db": 120
}
```

i If this returns an error, the backend server is down.

7. Quick Reference

Method	Endpoint	Purpose
GET	`/api/status`	Health check and pipeline status
GET	`/api/camera/stream`	MJPEG live feed (view only, no attendance)
WS	`/ws/camera`	WebSocket live feed + attendance marking
POST	`/api/camera/process-frame`	Submit single JPEG frame from edge camera
POST	`/api/register`	Register student with face photos
GET	`/api/students`	List active students
DELETE	`/api/students/{id}`	Deactivate a student (soft delete)
GET	`/api/attendance`	List attendance records (filterable)
GET	`/api/attendance/export`	Download attendance CSV
GET	`/api/stats`	Dashboard summary (present / absent / rate)

Notes for the development team

1. CORS is open -- all origins are allowed. No preflight issues.
2. All timestamps are in server local time. Convert to your timezone if needed.
3. All POST bodies use multipart/form-data (not JSON). File uploads require the photos field as an array of files.
4. The WebSocket binary frames are JPEG bytes. Draw them to a or use createObjectURL for display.
5. Swagger / interactive docs are available at: http://server:8000/docs